

Berufliche Hochschule Hamburg



Studiengang

Informatik B.Sc.

Cross-Site Scripting und Cross-Origin Resource Sharing im Scheduler-Projekt

Ausarbeitung im Rahmen des Moduls IT Security

Sommersemester 2026

Abgabe:

Hamburg, den 19. Juni 2026

Vorgelegt von:

Name	Matr.-Nr.	Anschrift	E-Mail
Linus Paliga	xxxxxx	Osterbrook 13c, 20537 Hamburg	l.paliga@stud.bhh.de
Raik Ole Kannenberg	xxxxxx	???	r.kannenberg@stud.bhh.de
Tom Markmann	227365	Lämmersieth 33, 22305 Hamburg	t.markmann@stud.bhh.de
Arvid Freese	xxxxxx	Albers-Schönberg-Stieg 21, 22307 Hamburg	a.freese@stud.bhh.de

Abstract

Dieser Projektbericht untersucht Cross-Site Scripting (XSS) und Cross-Origin Resource Sharing (CORS) anhand der realen Vue-/Spring-Boot-Anwendung *Scheduler*. Die Themen wurden innerhalb der Gruppe aufgeteilt: Arvid Freese und Linus Paliga bearbeiteten XSS, Tom Markmann und Raik Ole Kannenberg CORS. Nach der Herleitung der theoretischen Grundlagen wurde das Scheduler-Repository systematisch auf relevante Datenflüsse, Ausgabesenken und Sicherheitskonfigurationen geprüft. Für beide Themen entstanden anschließend lokale Live-Demos.

Die XSS-Analyse identifizierte eine reale Stored-DOM-XSS-Schwachstelle. Datenbankbasierte Veranstaltungsnamen, Gruppenbezeichnungen und Reservierungslabells wurden in mehreren Kalenderansichten an den Standardrenderer der Bibliothek `vue-cal` übergeben, der Event-Titel und -Inhalte als HTML rendert. Die Schwachstelle wurde durch eigene Event-Slots mit Vue-Interpolation strukturell behoben. Zum Berichtsstand ist dieser Fix auf `capstone-main` in allen relevanten Kalenderansichten ausgerollt; nach erneuter Prüfung besteht kein bekanntes aktuelles XSS-Risiko dieser Art.

Die Spring-API besaß im untersuchten Ausgangsstand keine pauschale CORS-Freigabe. Die normale lokale Anwendung vermeidet Cross-Origin-Aufrufe über den Vite-Proxy. Für die Demo wurde im separaten Security-Branch bewusst eine unsichere globale Origin-Freigabe ergänzt und anschließend einer festen Allowlist gegenübergestellt. Unabhängig davon bleiben konkrete Verbesserungen erforderlich: `/files/**` ist zu breit öffentlich freigegeben, Keycloak verwendet für den Web-Client eine Wildcard-Origin sowie nicht benötigte OAuth-Flows, und das Frontend setzt trotz Bearer-Token-Authentifizierung `credentials: include`. Der Bericht leitet daraus priorisierte, direkt auf Scheduler-Dateien bezogene Maßnahmen ab.

Inhaltsverzeichnis

1. Einleitung	1
1.1. Motivation	1
1.2. Zielsetzung und Leitfragen	1
1.3. Abgrenzung und Berichtsstand	1
2. Theoretische Grundlagen	2
2.1. Cross-Site Scripting	2
2.2. Quellen, Senken und sichere Ausgabe	2
2.3. Same-Origin Policy und CORS	3
2.4. CORS ist keine Authentifizierung	3
3. Projektvorgehen und Untersuchungsgegenstand	3
3.1. Arbeitsteilung	3
3.2. Scheduler-Architektur	4
3.3. Prüfmethode	4
4. XSS-Analyse und Behebung	5
4.1. Gefundene Stored-DOM-XSS-Schwachstelle	5
4.2. Konkreter Datenfluss und Risiko	5
4.3. Struktureller Fix	6
4.4. Bewertung nach dem Rollout	6
4.5. Verbleibende Schutzmaßnahmen	6
5. CORS-Analyse und Verbesserungen	6
5.1. Ausgangssituation der Spring-API	6
5.2. Bewusst unsichere Demo-Konfiguration	7
5.3. Sichere Vergleichskonfiguration	7
5.4. Öffentliche Datei-Endpunkte	7
5.5. Keycloak- und Frontend-Konfiguration	8
5.6. Bewertung	8
6. Live-Demos und technische Umsetzung	8
6.1. XSS-Demo	8
6.2. CORS-Demo	9
6.3. Aussagekraft und Grenzen	10
7. Bewertung und priorisierte MaSSnahmen	10
7.1. Ergebnisübersicht	10
7.2. Priorität 1: Öffentliche Dateioberfläche begrenzen	10
7.3. Priorität 2: XSS-Fix durch Tests absichern	11
7.4. Priorität 3: Keycloak und OAuth härten	11
7.5. Priorität 4: CORS nur bei fachlichem Bedarf aktivieren	11
7.6. Priorität 5: Zusätzliche Schutzschichten	11

8. Fazit	11
Literatur	13
A. Anhang	14
A.1. Zentrale Projektartefakte	14
A.2. Prüfliste für zukünftige Änderungen	14

1. Einleitung

1.1. Motivation

Webanwendungen verarbeiten fortlaufend Daten aus Formularen, Datenbanken, APIs und dem Browserkontext. Zwei zentrale Sicherheitsgrenzen sind dabei die Trennung von Daten und ausführbarem Code sowie die Trennung verschiedener Web-Origins. XSS verletzt die erste Grenze: Ein eigentlich als Text gedachter Wert wird als HTML oder JavaScript interpretiert. CORS steuert die zweite Grenze: Ein Server kann dem Browser erlauben, Antworten auch fremden Origins zugänglich zu machen.

Der Scheduler eignet sich als praxisnahes Untersuchungsobjekt, weil er zahlreiche nutzerkontrollierbare Stammdaten und Kalenderansichten, ein Vue-Frontend, eine Spring-Boot-API und Keycloak zur Authentifizierung kombiniert. Ein Fehler an der Ausgabegrenze kann dadurch im Browser eines eingeloggten Nutzers ausgeführt werden. Eine zu breite Origin-Freigabe kann wiederum die durch die Same-Origin Policy vorgesehene Lesesperre aufheben.

1.2. Zielsetzung und Leitfragen

Ziel des Projekts war nicht eine generische Darstellung der beiden Themen, sondern eine konkrete Sicherheitsprüfung des Scheduler-Projekts mit nachvollziehbaren Live-Demos und umsetzbaren Verbesserungen. Der Bericht beantwortet vier Leitfragen:

1. Welche XSS- und CORS-relevanten Datenflüsse und Konfigurationen existieren im Scheduler?
2. Welche Schwachstellen oder riskanten Konfigurationen lassen sich tatsächlich belegen?
3. Wie können Ursache, Wirkung und Behebung lokal demonstriert werden?
4. Welche MaSSnahmen sollten auf `capstone-main` dauerhaft umgesetzt und getestet werden?

1.3. Abgrenzung und Berichtsstand

Die Prüfung bezog sich auf getrackte Quell- und Konfigurationsdateien aus `api/`, `web/`, Docker, Nginx und dem Keycloak-Realm-Export `scheduler.json`. Die konkrete Rendering-Stelle in `vue-cal` wurde zusätzlich in der installierten Bibliothek untersucht. Produktive Systeme wurden nicht aktiv angegriffen.

Der Bericht beschreibt den Stand nach dem vollständigen Rollout des XSS-Fixes auf `capstone-main`. Die Demo-Setups wurden dagegen in einem separaten, von `capstone-main` abgeleiteten Security-Demobranch umgesetzt. Absichtlich schwache CORS-Einstellungen, Umschaltlogik und Demo-Testdaten gehören ausschliesslich zu diesem Demonstrationsstand und nicht zum produktiven Fix.

2. Theoretische Grundlagen

2.1. Cross-Site Scripting

XSS entsteht, wenn nicht vertrauenswürdige Daten eine Ausgabesenke erreichen, die den Wert als HTML oder JavaScript interpretiert. Der eingeschleuste Code läuft anschließend im Origin der legitimen Anwendung und unterliegt damit nicht der Same-Origin-Sperre gegenüber dieser Anwendung. Er kann sichtbare Inhalte lesen, die Oberfläche verändern und Requests über den vorhandenen Anwendungskontext auslösen. OWASP empfiehlt deshalb kontextgerechtes Output Encoding und sichere Framework-Ausgabe als primäre Schutzmaßnahme [4].

Die klassischen Varianten unterscheiden sich durch Herkunft und Lebensdauer des Payloads:

- **Stored XSS:** Der Payload wird persistent gespeichert und bei späteren Aufrufen erneut ausgeliefert.
- **Reflected XSS:** Der Payload stammt aus dem aktuellen Request und wird unmittelbar zurückgegeben.
- **DOM-based XSS:** Die unsichere Verarbeitung erfolgt im Browser, beispielsweise über `innerHTML`.

Der im Scheduler gefundene Fall kombiniert Stored und DOM-based XSS: Der Wert liegt dauerhaft in der Datenbank, wird über die API geladen und erst beim Rendern durch `vue-cal` als HTML in das DOM geschrieben.

2.2. Quellen, Senken und sichere Ausgabe

Eine *Quelle* ist ein kontrollierbarer Datenursprung, beispielsweise ein Veranstaltungsname, Gruppenname oder Reservierungslabel. Eine *Senke* ist eine Operation, die Daten als aktiven Inhalt interpretiert. Typische Senken sind `innerHTML`, `Vue-v-html`, `insertAdjacentHTML` oder ein Bibliotheksrenderer mit vergleichbarem Verhalten.

Vue-Interpolation mit doppelten geschweiften Klammern escaped HTML standardmäßig und erzeugt Text statt aktiver DOM-Strukturen [6]. Genau diese Eigenschaft wird beim Scheduler-Fix genutzt:

Listing 1: Sichere Ausgabe eines Kalenderereignisses

```
1 <template v-slot:event="{ event }">
2   <div class="vuecal__event-content">
3     <strong>{{ event.title }}</strong>
4     <div>{{ event.content }}</div>
5   </div>
6 </template>
```

Eine Content Security Policy (CSP) ist eine zusätzliche Schutzschicht, ersetzt aber nicht die Behebung der unsicheren Senke [3]. Serverseitige Validierung verbessert Datenqualität und begrenzt Payloads, kann kontextgerechtes Escaping ebenfalls nicht ersetzen.

2.3. Same-Origin Policy und CORS

Eine Origin besteht aus Schema, Host und Port. Deshalb sind `http://localhost:8078` und `http://localhost:8079` verschiedene Origins. Die Same-Origin Policy verhindert grundsätzlich, dass JavaScript einer Origin Antworten einer fremden Origin liest. Ein Request kann dabei teilweise trotzdem gesendet werden; blockiert wird vor allem der Zugriff des Skripts auf die Antwort.

CORS ist die kontrollierte Lockerung dieser Browserregel. Der Server liefert dafür unter anderem folgende Header [2]:

- `Access-Control-Allow-Origin`: erlaubte lesende Origin,
- `Access-Control-Allow-Credentials`: Freigabe browserseitiger Credentials,
- `Access-Control-Allow-Methods`: erlaubte HTTP-Methoden,
- `Access-Control-Allow-Headers`: erlaubte Request-Header,
- `Access-Control-Expose-Headers`: für JavaScript lesbare Response-Header.

Requests mit beispielsweise `Authorization` oder `Content-Type: application/json` lösen üblicherweise einen Preflight-Request mit `OPTIONS` aus. CORS muss deshalb vor der Spring-Security-Authentifizierung verarbeitet werden, da der Preflight noch keine Benutzer-Credentials enthält [5].

2.4. CORS ist keine Authentifizierung

CORS vergibt keine fachlichen Rechte und schützt eine API nicht vor direkten Requests außerhalb des Browsers. Authentifizierung beantwortet, wer ein Nutzer ist; Autorisierung beantwortet, was dieser Nutzer tun darf. CORS beantwortet lediglich, ob fremdes Browser-JavaScript eine Antwort lesen darf.

Ebenso wichtig ist die Abgrenzung beim Scheduler: Das Frontend setzt den Bearer-Token explizit im `Authorization`-Header. Eine fremde Webseite erhält diesen Token nicht automatisch allein durch eine offene CORS-Konfiguration. Ein Token-Leak setzt eine zusätzliche Schwachstelle voraus, etwa XSS, eine unsichere Token-Antwort oder falsch eingesetzte Cookie-Authentifizierung.

3. Projektvorgehen und Untersuchungsgegenstand

3.1. Arbeitsteilung

Die Bearbeitung wurde entlang der beiden Themen aufgeteilt:

Tabelle 1: Arbeitsteilung innerhalb der Projektgruppe

Thema	Verantwortliche und Aufgaben
XSS	Arvid Freese und Linus Paliga: Grundlagen, Suche nach Quellen und Senken, Analyse der <code>vue-cal</code> -Schwachstelle, XSS-Demo und Fix
CORS	Tom Markmann und Raik Ole Kannenberg: Grundlagen, Analyse von Spring Security, Vite, Keycloak und Datei-Endpunkten, CORS-Demo und sichere Konfiguration

Anders als in einem iterativen Issue-Prozess wurden zunächst die theoretischen Grundlagen getrennt erarbeitet. AnschlieSSend prüften beide Teilgruppen den Scheduler aus ihrer jeweiligen Perspektive. Die Ergebnisse wurden in den Markdown-Berichten `xss-analyse-und-demo.md` und `cors-analyse-und-demo.md` konsolidiert und für die gemeinsame Präsentation zusammengeführt.

3.2. Scheduler-Architektur

Der untersuchte Scheduler verwendet Spring Boot 3.0.1 und Java 17 im Backend sowie Vue 3 und Vite im Frontend. Die Authentifizierung erfolgt über Keycloak; die API validiert Bearer-JWTs als OAuth2 Resource Server.

Tabelle 2: Lokale Komponenten und Origins

Komponente	Adresse	Rolle
Keycloak	<code>http://localhost:8080</code>	Identitätsprovider
Spring-API	<code>http://localhost:8079</code>	Backend und Datenzugriff
Vue-Frontend	<code>http://localhost:8078</code>	reguläre Webanwendung
PostgreSQL	<code>localhost:5432</code>	persistente Daten
Demo-Seite	<code>http://localhost:5173</code>	fremde Origin für CORS-Demo

Das Frontend ruft die API relativ über `/api/...` auf. `web/vite.config.js` leitet diese Requests lokal an Port 8079 weiter. Aus Sicht des Browsers bleiben die normalen Requests damit same-origin zu Port 8078.

3.3. Prüfmethode

Die Analyse kombinierte statische Quellcodeprüfung, Konfigurationsvergleich und kontrollierte lokale Demonstrationen. Für XSS wurden nutzerkontrollierbare Felder bis zu ihren Ausgabestellen verfolgt und eigene sowie externe HTML-Senken gesucht. Für CORS wurden Spring Security, Vite-Proxy, Frontend-Requests, Keycloak-Web-Origins, Nginx und öffentliche Datei-Endpunkte geprüft.

Die weiteren relevanten Entwicklungsbranches wurden ebenfalls auf das VueCal-Muster geprüft; die ursprüngliche Schwachstelle ist dort vorhanden. Der dauerhafte Fix wurde jedoch ausschlieSSlich auf `capstone-main` ausgerollt, und zwar ohne Demo-Umschaltung und ohne

schwache CORS-Konfiguration. Der Security-Branch behält dagegen bewusst sichere und unsichere Varianten für den direkten Vergleich.

4. XSS-Analyse und Behebung

4.1. Gefundene Stored-DOM-XSS-Schwachstelle

Im eigenen Frontend-Code wurden keine direkten produktiven Verwendungen von `v-html`, `innerHTML`, `insertAdjacentHTML` oder `eval` gefunden. Die relevante Senke lag in der Drittbibliothek `vue-cal` in Version `^4.8.1`. Deren Standardrenderer behandelt `event.title` und `event.content` als HTML.

Mehrere Scheduler-Ansichten übergaben datenbankbasierte Werte an genau diese Felder, ohne einen eigenen Event-Slot zu definieren. Besonders relevant waren folgende Komponenten:

- `ReserveTimeslot.vue`: Reservierungslabel und Veranstaltungsname als `title`,
- `AssignmentView.vue`: Veranstaltungs- und Gruppenname als `title` beziehungsweise `content`,
- `EditIncompleteStateAvailabilities.vue`: Gruppenname als `content`,
- `TimeSlotCalendarView.vue`: Verwendung des Standardrenderers und HTML-Strings für Statusicons.

Die bereits vorhandene Datei `ScheduleOverview.vue` zeigte das sichere Gegenmuster: Sie definierte eigene `event`-Slots und gab Titel, Inhalte und Annotationen über Vue-Interpolation aus.

4.2. Konkreter Datenfluss und Risiko

Der nachgewiesene Datenfluss verlief über eine persistente Quelle und eine DOM-Senke:

Eingabefeld / Import → Datenbank → API → `event.title/content` → `vue-cal`
`innerHTML`

Ein harmloser lokaler Nachweis verwendete einen Wert der Form:

Listing 2: Lokaler XSS-Demo-Payload

```
1 XSS Demo <img src=x onerror="alert('XSS Demo')">
```

Das nicht vorhandene Bild löst den `onerror`-Handler aus. Der Alert beweist lediglich Codeausführung. Realistisch könnte eingeschleuster Code sichtbare Planungsdaten lesen, UI-Inhalte manipulieren oder API-Requests im Kontext des eingeloggtten Nutzers auslösen. Das Risiko wird dadurch erhöht, dass `web/src/App.vue` die Keycloak-Instanz als `window.$keycloak` verfügbar macht und der Access Token zusätzlich im Vuex-Session-State liegt.

4.3. Struktureller Fix

Die Schwachstelle wurde nicht durch eine Payload-Blacklist behoben, sondern durch Entfernung der unsicheren Ausgabesenke. Alle relevanten `VueCal`-Instanzen mit nutzer- oder datenbankkontrollierten Werten besitzen zum Berichtsstand eigene Event-Slots. Titel, Inhalte und Statusinformationen werden darin als Text oder als strukturierte UI-Elemente gerendert.

Der Security-Branch demonstriert den Unterschied exemplarisch mit zwei Komponenten:

- `ReserveTimeslot.vue`: ursprünglicher Standardrenderer für die unsichere Demo,
- `ReserveTimeslotSafe.vue`: eigener `v-slot:event` mit `{{ event.title }}`.

`web/.env.xss-safe`, die Umschaltung in `web/src/router.js` und die npm-Skripte `dev:xss-safe` beziehungsweise `build:xss-safe` sind reine Demo-Artefakte. Auf `capstone-main` ist der sichere Renderer direkt in den regulären Ansichten umgesetzt; eine Laufzeitumschaltung zurück zur anfälligen Variante ist dort nicht vorhanden.

4.4. Bewertung nach dem Rollout

Nach dem vollständigen Rollout und einer erneuten Prüfung aller relevanten Kalenderansichten auf `capstone-main` besteht dort kein bekanntes aktuelles XSS-Risiko über den `vue-cal`-Standardrenderer. Die weiteren relevanten Entwicklungsbranches wurden ebenfalls geprüft und enthalten die ursprüngliche Schwachstelle weiterhin; der Fix wurde bewusst nur auf `capstone-main` ausgerollt. Diese Aussage bedeutet nicht, dass zukünftige Änderungen automatisch sicher bleiben. Neue Bibliotheken, neue Kalenderansichten oder eine spätere Verwendung von `v-html` können dieselbe Schwachstellenklasse erneut einführen.

4.5. Verbleibende Schutzmaßnahmen

Für nachhaltigen Schutz sind folgende Ergänzungen sinnvoll:

- Security-Regressionstests, die typische Payloads in jeder Kalenderansicht als Text erwarten,
- serverseitige Längen- und Fachvalidierung für Namen, Labels, Annotationen und Freitextfelder,
- eine produktionsgerechte CSP ohne unnötiges `unsafe-inline`,
- regelmäßige Prüfung von Dependency-Updates und Bibliotheksrenderern,
- Entfernung unnötiger globaler Token-Zugriffe wie `window.$keycloak`.

5. CORS-Analyse und Verbesserungen

5.1. Ausgangssituation der Spring-API

Im Ausgangsstand von `capstone-main` war keine explizite Spring-CORS-Konfiguration vorhanden. Es gab weder `@CrossOrigin` noch eine `CorsConfigurationSource`, einen

`CorsFilter` oder `http.cors()`. Direkte Cross-Origin-Antworten der API waren damit für fremdes Browser-JavaScript standardmäßig nicht lesbar.

Für die normale lokale Anwendung ist dies funktional unproblematisch. Die Vite-Konfiguration proxyt relative `/api`-Requests von Port 8078 an die API auf Port 8079. Der Browser sieht nur die Frontend-Origin. CORS wird erst sichtbar, wenn eine separate Seite, etwa auf Port 5173, die API direkt anspricht.

5.2. Bewusst unsichere Demo-Konfiguration

Der Branch `its-xss-cors-security-break` ergänzt für die Live-Demo eine Spring-CORS-Bean vom Typ `CorsConfigurationSource`. Die unsichere Variante akzeptiert mit `allowedOriginPatterns("*")` jede Origin und erlaubt gleichzeitig Credentials. Zusätzlich werden sämtliche üblichen Methoden und alle Header global für `/**` freigegeben.

Listing 3: Bewusst unsichere CORS-Demo, nicht für Produktion

```
1 config.setAllowedOriginPatterns(List.of("*"));
2 config.setAllowCredentials(true);
3 config.setAllowedMethods(List.of(
4     "GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
5 config.setAllowedHeaders(List.of("*"));
6 source.registerCorsConfiguration("/**", config);
```

Diese Konfiguration spiegelt praktisch jede anfragende Origin und hebt die Browser-Lesesperre global auf. Sie ist kein Bestandteil des produktiven XSS-Fixes und darf nicht nach `capstone-main` übernommen werden.

5.3. Sichere Vergleichskonfiguration

Die sichere Demo-Variante erlaubt ausschließlich das reguläre lokale Frontend:

Listing 4: Restriktive lokale Vergleichskonfiguration

```
1 config.setAllowedOrigins(List.of("http://localhost:8078"));
2 config.setAllowCredentials(false);
```

Eine dauerhafte CORS-Konfiguration sollte nur eingeführt werden, wenn ein echtes Cross-Origin-Frontend benötigt wird. Dann müssen Origins aus umgebungsspezifischen Properties stammen, Methoden und Header auf den tatsächlichen Bedarf begrenzt und Routen möglichst selektiv registriert werden. Da der Scheduler Bearer-Tokens explizit im `Authorization`-Header sendet, ist `allowCredentials(true)` für den aktuellen API-Flow nicht erforderlich.

5.4. Öffentliche Datei-Endpunkte

Die wichtigste CORS-nahe, aber eigenständige Schwachstelle liegt in `api/src/main/java/com/frostbear/scheduler/conf/SecurityConfiguration.java`. Der Matcher

```
1 requestMatchers("/files/**").permitAll();
```

macht nicht nur bewusst tokenisierte Download-Links öffentlich, sondern auch den Voll-Export `GET /files/planningperiod/{id}/download` sowie mehrere Endpunkte zur Token-Erzeugung im `FileController`. CORS schützt diese Daten nicht vor direktem Zugriff. Die

Demo zeigt lediglich zusätzlich, dass eine breite CORS-Freigabe die Antwort auch für fremdes Browser-JavaScript lesbar macht.

Die Regel sollte auf tatsächlich öffentliche Token-Downloads begrenzt werden. Voll-Exporte und Token-Erzeugung müssen authentifiziert, rollenbasiert autorisiert und auf die konkrete Planungsperiode beziehungsweise Person beschränkt werden.

5.5. Keycloak- und Frontend-Konfiguration

Der Realm-Export `scheduler.json` enthält für den Web-Client eine zu breite Konfiguration:

- `webOrigins: ["*"],`
- `implicitFlowEnabled: true,`
- `directAccessGrantsEnabled: true.`

Zusätzlich setzt der Realm `sslRequired` auf `none` und deaktiviert den Brute-Force-Schutz. Der OAuth 2.0 Security Best Current Practice rät vom Implicit Grant und vom Resource Owner Password Credentials Grant ab [1]. Für den Scheduler-Web-Client sollten konkrete Origins, Authorization Code Flow mit PKCE, TLS außerhalb der lokalen Entwicklung und aktivierter Login-Schutz verwendet werden.

`web/src/shared/http-client.js` setzt bei jedem Request `credentials: 'include'`, obwohl die API mit einem expliziten Bearer-Token arbeitet. Der Wert sollte auf `same-origin` oder `omit` reduziert werden, solange keine Cookie-basierte API-Sitzung benötigt wird. Dies verhindert, dass spätere Cookies unbeabsichtigt an Cross-Origin-Ziele gesendet werden.

5.6. Bewertung

Die Spring-API besaß im sicheren Ausgangsstand keine klassische Allow-All-CORS-Lücke. Das aktuelle Risiko entsteht aus benachbarten Konfigurationen: zu breite öffentliche Datei-Endpunkte, Keycloak-Wildcards, nicht benötigte OAuth-Flows und unnötige Credential-Übermittlung. Werden diese Punkte mit einer später versehentlich global aktivierten CORS-Freigabe kombiniert, wächst die Angriffsfläche erheblich.

6. Live-Demos und technische Umsetzung

6.1. XSS-Demo

Die XSS-Demo stellt denselben persistent gespeicherten Wert in einer anfälligen und einer sicheren Kalenderansicht gegenüber. Der Test `api/src/test/java/com/frostbear/scheduler/demo/XssDemoSetupTest.java` bereitet eine vorhandene lokale Planungsperiode mit reproduzierbaren Demo-Daten vor. Die zugehörige Gradle-Aufgabe `prepareXssDemo` und die Property `xssDemoPlanningPeriodId` erlauben die gezielte Auswahl.

Die unsichere Ansicht übergibt das Event vollständig an den Standardrenderer von `vue-cal`. Die sichere Ansicht `ReserveTimeslotSafe.vue` definiert einen eigenen Event-Slot. Über

VITE_XSS_SAFE_RESERVATION_VIEW und die Router-Umschaltung kann für die Präsentation zwischen beiden Darstellungen gewechselt werden.

Der entscheidende Vergleich ist nicht ein unterschiedlicher Payload, sondern dieselbe gespeicherte Eingabe:

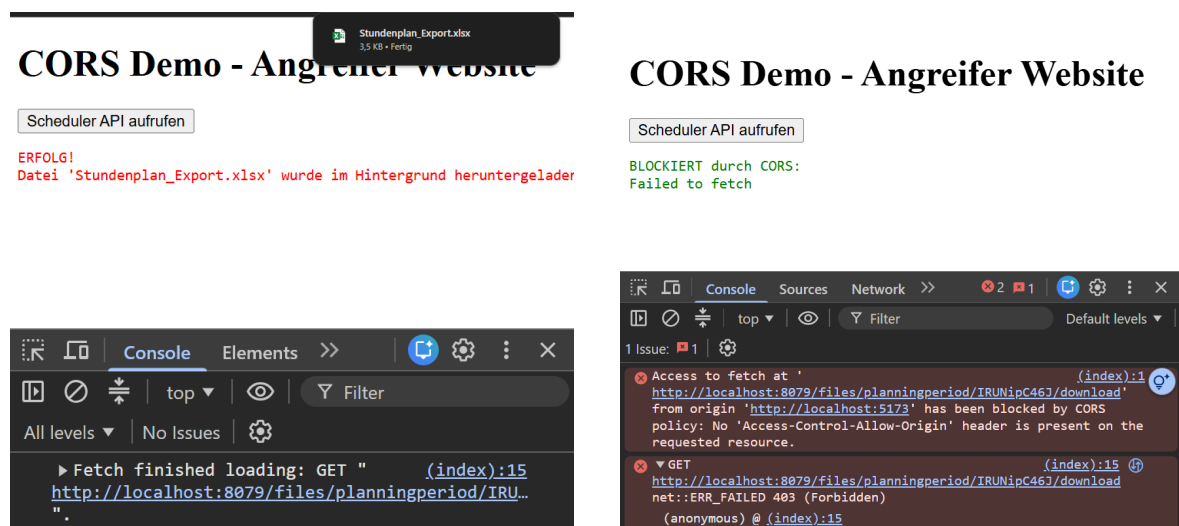
1. Unsichere Ansicht öffnen: Der Browser interpretiert den Wert als HTML und der lokale Alert erscheint.
2. Sichere Ansicht öffnen: Der vollständige Payload bleibt sichtbar, wird aber nur als Text dargestellt.
3. Ursache im Code zeigen: Standardrenderer mit HTML-Senke gegenüber eigenem Slot mit Vue-Interpolation.

6.2. CORS-Demo

Die CORS-Demo verwendet vier lokale Komponenten: Keycloak und PostgreSQL, die Spring-API auf Port 8079, das reguläre Vue-Frontend auf Port 8078 sowie die separate Angreifer-Demo aus `cors/index.html` auf Port 5173.

Die Seite ruft den öffentlichen Planungsperioden-Export direkt auf und versucht, die Antwort als Datei zu lesen. Der Vergleich zeigt:

1. Ohne passende CORS-Freigabe oder mit fester Allowlist blockiert der Browser den Lesezugriff der Origin `localhost:5173`.
2. Mit der bewusst unsicheren globalen Freigabe kann die fremde Seite die Antwort lesen und den Download auslösen.



Unsichere globale Freigabe: Antwort ist lesbar.

Feste Allowlist: Demo-Origin wird blockiert.

Abbildung 1: Gegenüberstellung der lokalen CORS-Demo

6.3. Aussagekraft und Grenzen

Die Demos beweisen jeweils einen klar abgegrenzten Mechanismus. Der XSS-Alert beweist Codeausführung im Scheduler-Origin, aber noch keine konkrete Datenexfiltration. Der CORS-Dateidownload beweist, dass eine fremde Origin eine öffentliche Antwort lesen kann, aber noch keinen Token-Diebstahl.

Insbesondere wird der Bearer-Token des Scheduler-Frontends nicht automatisch durch `allowCredentials(true)` an die Demo-Origin übertragen. Ein vollständiger Account-Takeover würde einen zusätzlichen Fehler voraussetzen, etwa die nachgewiesene XSS-Klasse vor ihrem Fix, eine unsichere Token-Antwort oder eine spätere Cookie-basierte Authentifizierung mit fehlerhaftem CORS.

7. Bewertung und priorisierte MaSSnahmen

7.1. Ergebnisübersicht

Tabelle 3: Bewertung zum Berichtsstand nach XSS-Rollout

Bereich	Status	Begründung
VueCal Stored DOM XSS	behoben	Eigene Event-Slots und Vue-Interpolation sind auf <code>capstone-main</code> vollständig ausgerollt; erneute Prüfung ohne aktuellen Befund.
Eigene Vue-HTML-Senken	kein Befund	Keine produktive direkte Verwendung von <code>v-html</code> , <code>innerHTML</code> oder vergleichbaren Senken in <code>web/src</code> .
Spring API-CORS	kein Allow-All-Befund	Ohne Demo-Konfiguration keine explizite globale CORS-Freigabe; normale Entwicklung nutzt den Vite-Proxy.
<code>/files/**</code>	hoch	Voll-Exporte und Token-Erzeugung liegen unter einer globalen <code>permitAll</code> -Regel.
Keycloak Web-Client	hoch	Wildcard-Web-Origin und nicht benötigte beziehungsweise veraltete OAuth-Flows.
Security-Regression	ausbaufähig	Spezifische automatisierte XSS-, CORS- und Autorisierungstests fehlen im regulären Stand.

7.2. Priorität 1: Öffentliche Dateioberfläche begrenzen

Die pauschale Freigabe von `/files/**` sollte durch getrennte Regeln ersetzt werden. Nur ausdrücklich öffentliche, ausreichend zufällige und möglichst zeitlich begrenzte Token-Downloads dürfen ohne Login erreichbar sein. Folgende Endpunkte müssen mindestens authentifiziert und fachlich autorisiert werden:

- GET `/files/planningperiod/{id}/download`,
- sämtliche POST `/files/create/...`-Endpunkte,

- `POST /files/get-or-create-ics-token`.

7.3. Priorität 2: XSS-Fix durch Tests absichern

Der ausgerollte Fix sollte mit Komponenten- oder End-to-End-Tests gegen Regression geschützt werden. Für jede VueCal-Ansicht wird ein Event mit HTML- und Event-Handler-Zeichenfolge gerendert. Der Test muss bestätigen, dass der String als Text sichtbar bleibt und kein DOM-Element aus dem Payload entsteht. Ergänzend sollte CI nach neuen Verwendungen von `v-html`, `innerHTML` und ähnlichen Senken suchen.

7.4. Priorität 3: Keycloak und OAuth härten

In `scheduler.json` sind konkrete Frontend-Origins statt `*` zu hinterlegen. Implicit Flow und Direct Access Grants sollten für den SPA-Web-Client deaktiviert, der Authorization Code Flow mit PKCE verwendet und der Brute-Force-Schutz aktiviert werden. Außerhalb lokaler Entwicklung muss TLS erzwungen werden.

7.5. Priorität 4: CORS nur bei fachlichem Bedarf aktivieren

Solange Frontend und API über denselben Origin beziehungsweise Reverse Proxy erreichbar sind, ist keine globale API-CORS-Freigabe nötig. Falls Cross-Origin-Zugriff eingeführt wird, sollte `SecurityConfiguration.java` eine properties-basierte Allowlist mit blockierendem Default erhalten. Die Registrierung sollte auf tatsächlich benötigte Routen begrenzt werden. Demo-Origins wie `localhost:5173` dürfen ausschliesslich in einem lokalen Entwicklungsprofil vorkommen.

7.6. Priorität 5: Zusätzliche Schutzschichten

- `credentials: 'include'` in `web/src/shared/http-client.js` auf `same-origin` oder `omit` reduzieren.
- CSP im produktiven Reverse Proxy oder Backend einführen und zunächst im Report-Only-Modus testen.
- Freitextfelder serverseitig auf fachlich sinnvolle Längen und Werte validieren.
- `window.$keycloak` entfernen oder den globalen Zugriff mindestens reduzieren.
- Dependency- und Security-Checks für npm und Gradle in CI integrieren.
- Die produktive Nginx-/Ingress-Konfiguration dokumentieren; die eingetragene `nginx.conf` enthält derzeit keine `/api`-Proxy-Regel.

8. Fazit

Die Untersuchung zeigt, dass Sicherheitsprobleme im Scheduler nicht nur durch offensichtlich unsicheren eigenen Code entstehen. Die reale XSS-Schwachstelle lag im Zusammenspiel

aus kontrollierbaren Datenbankwerten und dem komfortablen, aber für diese Daten ungeeigneten Standardrenderer von `vue-cal`. Der strukturelle Fix über eigene Event-Slots und Vue-Interpolation beseitigt die Ursache. Nach dem vollständigen Rollout auf `capstone-main` und erneuter Prüfung besteht kein bekanntes aktuelles XSS-Risiko dieser Art.

Bei CORS war die Ausgangslage differenzierter: Die Spring-API war nicht pauschal für fremde Origins freigegeben und die normale lokale Anwendung vermeidet Cross-Origin-Aufrufe über den Vite-Proxy. Erst die bewusst unsichere Demo-Konfiguration machte den öffentlichen Datei-Export für eine fremde Browser-Origin lesbar. Die Demo verdeutlicht damit korrekt die Wirkung von CORS, darf aber nicht mit Authentifizierung oder einem automatischen Token-Leak gleichgesetzt werden.

Der größtenteils verbleibende Handlungsbedarf liegt in den angrenzenden Schutzschichten. Die globale `/files/**`-Freigabe muss begrenzt, die Keycloak-Konfiguration gehärtet und der XSS-Fix durch Regressionstests abgesichert werden. CORS sollte weiterhin blockierend bleiben, solange kein fachlicher Cross-Origin-Bedarf existiert. Die zentrale Erkenntnis aus beiden Themen ist gleich: Sicherheitsgrenzen müssen explizit kontrolliert werden. Nutzerdaten gehören an eine Textausgabegrenze; fremde Origins erhalten nur die minimal erforderliche Lesefreigabe.

Literatur

- [1] Torsten Lodderstedt, John Bradley, Andrey Labunets, and Daniel Fett. Best current practice for oauth 2.0 security. Technical Report RFC 9700, Internet Engineering Task Force, 2025.
- [2] Mozilla Developer Network. Cross-origin resource sharing (cors). <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS>, 2026. Abgerufen am 11. Juni 2026.
- [3] OWASP Foundation. Content security policy cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html, 2026. Abgerufen am 11. Juni 2026.
- [4] OWASP Foundation. Cross site scripting prevention cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html, 2026. Abgerufen am 11. Juni 2026.
- [5] Spring. Cors. <https://docs.spring.io/spring-security/reference/servlet/integrations/cors.html>, 2026. Abgerufen am 11. Juni 2026.
- [6] Vue.js. Security. <https://vuejs.org/guide/best-practices/security.html>, 2026. Abgerufen am 11. Juni 2026.

A. Anhang

A.1. Zentrale Projektartefakte

Tabelle 4: Relevante Artefakte für Analyse und Reproduktion

Artefakt	Pfad
Gemeinsame Präsentation	XSS-CORS-Praesentation.pptx
XSS-Analyse	xss/xss-analyse-und-demo.md
CORS-Analyse	cors/cors-analyse-und-demo.md
CORS-Demo-Seite	cors/index.html
CORS-Screenshots	cors/Corssecure.png, cors/Corsunsecure.png
Security-Demobranch	its-xss-cors-security-break
XSS-Demo-Setup	api/src/test/java/com/frostbear/scheduler/demo/ XssDemoSetupTest.java
Sichere Demo-Ansicht	web/src/admin/views/data-admin/PlanningPeriodCrud/ ReserveTimeslotSafe.vue
Spring Security	api/src/main/java/com/frostbear/scheduler/conf/ SecurityConfiguration.java
Datei-Endpunkte	api/src/main/java/com/frostbear/scheduler/admin/ controller/FileController.java
Keycloak-Realm	scheduler.json
Frontend-HTTP-Client	web/src/shared/http-client.js
Vite-Proxy	web/vite.config.js

A.2. Prüfliste für zukünftige Änderungen

1. Enthält jede neue VueCal-Ansicht einen eigenen sicheren Event-Slot?
2. Werden nutzerkontrollierte Werte ausschliesslich über Interpolation als Text ausgegeben?
3. Gibt es neue Verwendungen von `v-html`, `innerHTML`, Raw HTML, Markdown-Renderern oder HTML-Tooltips?
4. Sind neue Freitextfelder serverseitig validiert und durch Rendering-Tests abgedeckt?
5. Ist eine neue CORS-Origin fachlich erforderlich und umgebungsspezifisch konfiguriert?
6. Bleiben CORS-Methoden, Header und Routen auf das Minimum begrenzt?
7. Ist jeder neue `/files`-Endpunkt ausdrücklich authentifiziert und autorisiert oder bewusst als öffentlicher Token-Download dokumentiert?
8. Werden Keycloak-Web-Origins und OAuth-Flows bei Änderungen erneut geprüft?